

Report - **Project 2**

DETI SHOP

Course: SIO -

Date: Aveiro, 05/09/2024

Student: 106078 : João Vitor B. D. Ferreira

1- Introduction

Deti Shop is a web application where users can buy memorabilia from DETI (Department of Electronics, Telecommunications, and Informatics) at the University of Aveiro. The online shop has a variety of items, including mugs, cups, t-shirts, hoodies

2- System description

This project is a web-based e-commerce platform built using **Flask**. It enables users to register accounts, log in, manage shopping carts, place orders, manage user profiles, and upload profile images. The application employs **SQLite** as its database, and it follows the **blueprint architecture** and **application factory pattern** of Flask to organize the various modules of the system. The system was structured to be modular, making it easier to maintain and scale.

3- Project Overview

The goal of this project is to identify and rectify security flaws in the application, using the ASVS for guidance. After the audit it was possible to assure that the application **covers 50 out of 78** level 1 of the Application Security Verification Standard (ASVS) **criteria** and **12** requirements were considered **not applicable**.

After the modifications made to the application, the following ASVS requirements were satisfied:

1. Password Security Credentials (V2.1.1), (V2.1.2), (V2.1.3), (V2.1.4), (V2.1.9)
2. File Upload Requirements (V12.1.1)
3. General Authenticator Requirements (V2.2.1)
4. Password Security Credentials (V2.1.7)
5. Password Security Credentials (V2.1.8) **
6. Single or Multi Factor One Time Verifier Requirements ASVS V2.8 **

Two of three mandatory features were implemented according to project requirements:

1. **Multi-factor Authentication (MFA) - TOTP authentication login:**
2. **Password strength evaluation**

4- Overcome Vulnerabilities

4.1- Password Security Credentials (V2.1.1), (V2.1.2), (V2.1.3), (V2.1.4), (V2.1.9)

- V2.1.1 : Verify that user set passwords are at least 12 characters in length (after multiple spaces are combined).
- V2.1.2 : Verify that passwords 64 characters or longer are permitted but may be no longer than 128 characters
- V2.1.3 : Verify that password truncation is not performed. However, consecutive multiple spaces may be replaced by a single space
- V2.1.4 : Verify that any printable Unicode character, including language neutral characters such as spaces and Emojis are permitted in passwords
- V2.1.9 : Verify that there are no password composition rules limiting the type of characters permitted. There should be no requirement for upper or lower case or numbers or special characters

Code:

```
def validate_password(password, confirm_password):  
  
    if password != confirm_password:  
        flash('Passwords do not match. Please try again.', 'error')  
        return False  
  
    normalized_password = re.sub(r'\s+', ' ', password).strip()  
  
    if len(normalized_password) < 12:  
        flash("Password must be at least 12 characters long after combining multiple spaces.", "error")  
        return False  
  
    if len(normalized_password) > 128:  
        flash("Password cannot be more than 128 characters long.", "error")  
        return False  
  
    if not all(character.isprintable() for character in password):  
        flash("Password contains non-printable characters.", "error")  
        return False  
  
    return True
```

Explanation:

In the last version of the application, the verification method of the passwords were

upgraded and now follows ASVS requirements above.

The line: `normalized_password = re.sub(r'\s+', ' ', password).strip()` adjusts the password by replacing multiple consecutive whitespace characters with a single space and removing any leading or trailing whitespace. This ensures that the password length is accurately measured and any extra spaces do not affect validation.

The line: `if not all(character.isprintable() for character in password)` checks if every character in the password is printable. This ensures that the password only contains characters that are visible and usable in text.

Unlike the initial version, there are no password composition rules that limit the type of characters allowed.

4.2- File Upload Requirements (V12.1.1)

- *V12.1.1 : Verify that the application will not accept large files that could fill up storage or cause a denial of service.*

Previously, the size of uploaded files was not limited, which could cause performance issues for the application if the vulnerability was exploited by a malicious user.

Code:

```
class Config:
    SECRET_KEY = '@D3ti_SHop2024%('
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SQLALCHEMY_DATABASE_URI = f'sqlite:///{'os.path.join(basedir, "database.db")}'
    UPLOAD_FOLDER = 'static/img/profile_pics'
    MAX_CONTENT_LENGTH = 1 * 1024 * 1024 # 1 MB
```

app_sec

```
@errors.app_errorhandler(413)
def file_too_large(e):
    if request.endpoint == 'profile.upload_profile_picture':
        flash("File size must be less than 1 MB", 'error')
        return redirect(url_for('profile.profile_page'))
    elif request.endpoint == 'auth.register':
        flash("File size must be less than 1 MB", 'error')
        return redirect(url_for('auth.register'))

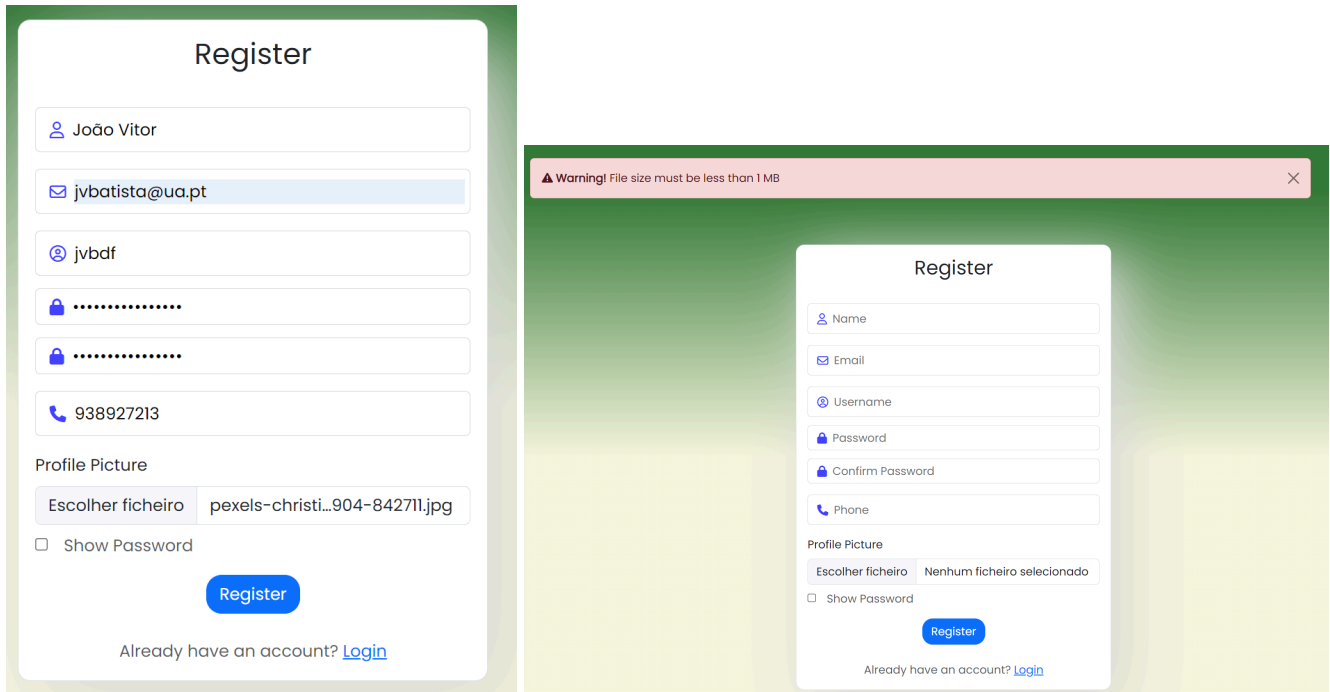
    return "File too large", 413
```

Explanation:

To fix this vulnerability, the maximum file size was set to 1MB. Flask's error handler was overwritten to handle exceptions conveniently.

Action:

A user tries to upload a file larger than 1 MB and receives an error message.



4.3-General Authenticator Requirements (V2.2.1)

- V2.2.1 : Verify that anti-automation controls are effective at mitigating breached credential testing, brute force, and account lockout attacks. Such controls include blocking the most common breached passwords, soft lockouts, rate limiting, CAPTCHA, ever increasing delays between attempts, IP address restrictions, or risk-based restrictions such as location, first login on a device, recent attempts to unlock the account, or similar. Verify that no more than 100 failed attempts per hour is possible on a single account.

Google Recaptcha v2 was used to prevent automated attacks.

Code:

```
<!-- reCAPTCHA widget -->
<div class="g-recaptcha" data-sitekey="6LcT7DcqAAAAAF7nP7GtBgnt1Yv7yhAr550CgEp"></div>
<br>
<div style="text-align: center;">
```

Recaptcha v2 widget was added to both login and register forms on submit.

```
def validate_captcha(captcha_response):
    payload = {
        'secret': Config.RECAPTCHA_SECRET_KEY ,
        'response': captcha_response
    }
    response = requests.post('https://www.google.com/recaptcha/api/siteverify', data=payload)
    result = response.json()
    return result.get('success', False)
```

Login

☐ Show Password

[Forgot password?](#)

☐ Não sou um robô



reCAPTCHA

Privacidade - Termos de Utilização

Don't have an account? [Sign up](#)

Register

Profile Picture

☐ Show Password

☐ Não sou um robô



reCAPTCHA

Privacidade - Termos de Utilização

Already have an account? [Login](#)

4.4- Password Security Credentials (V2.1.7)

- V2.1.7 :Verify that passwords submitted during account registration, login, and password change are checked against a set of breached passwords either locally (such as the top 1,000 or 10,000 most common passwords which match the system's password policy) or using an external API. If using an API a zero knowledge proof or other mechanism should be used to ensure that the plain text password is not sent or used in verifying the breach status of the password. If the password is breached, the application must require the user to set a new non-breached password.

Have I Been Pwned API was used to verify if the user's password is breached.

```
def is_password_breached(password):
    pwd_hash = hashlib.sha1(password.encode('utf-8')).hexdigest().upper()

    prefix = pwd_hash[:5]
    suffix = pwd_hash[5:]

    url = f"https://api.pwnedpasswords.com/range/{prefix}"

    response = requests.get(url)
    if response.status_code == 200:
        if suffix in response.text:
            return True
    else:
        flash("Error connecting to the breach database", 'error')
        return False
    return False
```

Action: A user tries to use a password such as “1234567891011”, and receives an warning that his password has been breached.

⚠ Warning! Password has been breached. Please choose a different password.

4.5- Password Security Credentials (V2.1.8)

- V2.1.8 :*"Verify that a password strength meter is provided to help users set a stronger password."*

A password strength meter was added to the registration form using Bootstrap classes and JavaScript to provide real-time feedback to users about the strength of their passwords.

Code:

```
function checkPasswordStrength() {
    var password = document.getElementById("password").value;
    var strengthBar = document.getElementById("password-strength-bar");
    var strengthText = document.getElementById("password-strength-text");
    var strength = 0;

    if (password.length >= 12) strength += 1; // Comprimento mínimo
    if (/[A-Z]/.test(password)) strength += 1; // Letra maiúscula
    if (/[a-z]/.test(password)) strength += 1; // Letra minúscula
    if (/[\d]/.test(password)) strength += 1; // Números
    if (/[\W_]/.test(password)) strength += 1; // Caracteres especiais

    switch (strength) {
        case 0:
        case 1:
            strengthBar.style.width = "20%";
            strengthBar.className = "progress-bar bg-danger";
            strengthText.textContent = "Very Weak";
            break;
        case 2:
            strengthBar.style.width = "40%";
            strengthBar.className = "progress-bar bg-warning";
            strengthText.textContent = "Weak";
            break;
        case 3:
            strengthBar.style.width = "60%";
            strengthBar.className = "progress-bar bg-info";
            strengthText.textContent = "Moderate";
            break;
        case 4:
            strengthBar.style.width = "80%";
            strengthBar.className = "progress-bar bg-primary";
            strengthText.textContent = "Strong";
            break;
        case 5:
            strengthBar.style.width = "100%";
            strengthBar.className = "progress-bar bg-success";
            strengthText.textContent = "Very Strong";
            break;
    }
}
```

Action:

As the user types their password, the JavaScript function `checkPasswordStrength()` dynamically updates the progress bar and a text label to indicate the password's strength, ranging from "Very Weak" to "Very Strong".

4.6-Single or Multi Factor One Time Verifier Requirements ASVS V2.8

- V2.8.1 : *Verify that time-based OTPs have a defined lifetime before expiring.*

The application implements a TOTP-based two-factor authentication system using the `pyotp` library. This library follows the TOTP standard (RFC 6238), which ensures that OTP codes are time-based and have a defined expiration period.

Each user is assigned a unique TOTP secret, which is generated using the `pyotp.random_base32()` function when the user enables two-factor authentication.

The application uses this secret to generate a provisioning URI, which is then converted into a QR code that can be scanned by the user's authentication app.

Code:

```
@auth.route('/enable_2fa')
def enable_2fa():
    user = User.query.filter_by(username=session['username']).first()

    if user.otp_enabled:
        flash('2FA is already enabled for your account.', 'error')
        return redirect(url_for('profile.profile_page'))

    secret = pyotp.random_base32()
    user.otp_secret = secret
    db.session.commit()

    # Generate a QR code for the user to scan
    totp = pyotp.TOTP(secret)
    qr_url = totp.provisioning_uri(user.username, issuer_name='DETI Shop')
    qr_img = qrcode.make(qr_url)

    # Convert the QR code to base64 for display
    buffered = BytesIO()
    qr_img.save(buffered, format="PNG")
    qr_b64 = base64.b64encode(buffered.getvalue()).decode()

    return render_template('enable_2fa.html', qr_b64=qr_b64)
```

```
@auth.route('/verify_totp', methods=['GET', 'POST'])
def verify_totp():
    if request.method == 'POST':

        totp_code = request.form['totp']
        user = User.query.filter_by(username=session['username']).first()

        # Verificar o código TOTP
        totp = pyotp.TOTP(user.otp_secret)

        if totp.verify(totp_code):
            login_user(user)
            session.pop('username', None)
            return redirect(url_for('products.products_page'))
        else:
            flash('INVALID TOTP CODE', 'error')
            return redirect(url_for('auth.verify_totp'))

    return render_template('verify_totp.html')
```

```
@auth.route('/login', methods=['GET', 'POST'])
def login():

    if session.get('user_id'):
        return redirect(url_for('products.products_page'))

    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        captcha_response = request.form['g-recaptcha-response']

        if validate_captcha(captcha_response):
            user = User.query.filter_by(email=email).first()

            if user.check_password(password):

                if user.otp_enabled and user.otp_secret :

                    session['username'] = user.username

                    return redirect(url_for('auth.verify_totp'))
                else:
                    session['username'] = user.username
                    return redirect(url_for('auth.enable_2fa'))

            else:
                flash('Login failed. Check your email and/or password.', 'error')
                return redirect(url_for('auth.login'))

    return render_template('login.html')
```

Example:

After correctly entering the access credentials, the user is redirected to a page where they must scan the QR code to link the totp to their favorite authentication application. Once this is done, upon entering the correct code, the login is effectively performed and the user is redirected to the store.

